Unix Commands Reference

A Comprehensive Guide to Essential Utilities • June 27, 2026

1. cal — Display Calendar

SYNTAX

`cal`

`cal year`

`cal month year`

WORKING

Without arguments, cal prints a calendar of the current month, with today's date usually highlighted. If you give it just a year (1–9999), it prints the calendar for the entire year. If you give it a month number (1–12) followed by a year, it prints the calendar of that specific month.

COMMONLY USED FORMS

- `cal` → current month
- `cal 2026` → full year calendar
- `cal 6 2026` → June 2026 only

EXAMPLE

```
$ cal 6 2026
      June 2026
Su Mo Tu We Th Fr Sa
    1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30
```

> **Analogy:** Like flipping through a desk calendar — flip to "this month" by default, or tell it exactly which month/year page to open.

2. date — Display/Set System Date and Time

SYNTAX

`date`

`date +format`

`date -u`

WORKING

Without arguments, prints the current system date and time using the system's default format (day, month, date, time, timezone, year). With a + followed by format specifiers, it prints only the parts you ask for, in the order and style you want. The -u flag shows the time in UTC (Universal Time) instead of local time.

COMMONLY USED FORMAT SPECIFIERS

Specifier	Meaning
%d	Day of month (01–31)
%m	Month number (01–12)
%y	Year, 2-digit
%Y	Year, 4-digit
%H	Hour (00–23)
%M	Minute
%S	Second
%A	Full weekday name
%B	Full month name

COMMONLY USED FLAG

- `-u` → display date/time in UTC

EXAMPLE

```
$ date
Sat Jun 27 14:32:10 IST 2026
```

```
$ date +%d-%m-%Y
27-06-2026

$ date +"%A, %B %d"
Saturday, June 27

$ date -u
Sat Jun 27 09:02:10 UTC 2026
```

> **Analogy:** Like a wall clock that, by default, shows you everything (day, date, time) — but you can also ask it "just tell me the day name" or "just tell me in GMT," and it'll answer only that part.

3. pr — Format Files for Printing

SYNTAX

`pr [options] filename`

WORKING

pr takes a plain text file and reformats it into paginated output — adding a header (filename, date, page number) at the top of each page, and splitting content into fixed-size pages — originally so it could be sent straight to a line printer.

COMMONLY USED FLAGS

Flag	Meaning
-l n	Set page length to n lines (default is 66)
-h "title"	Use a custom header text instead of filename
-d	Double-space the output
-n	Number each line
+page	Start printing from a given page number
-o n	Offset — indent every line by n spaces

EXAMPLE

```
$ pr -l 20 -d -n students.txt
```

This prints students.txt, paginated at 20 lines per page, double-spaced, with line numbers added.

> **Analogy:** Like a teacher taking a rough handwritten attendance list and typing it up neatly into printable pages, each with a heading and page number, ready to hand over to the office.

4. who — Show Who Is Logged In

SYNTAX

`who [options]`

WORKING

who reads the system's login records and prints the username, terminal line, and login time for every user currently logged in to the Unix system.

COMMONLY USED FLAGS

Flag	Meaning
-H	Print column headers (USER, LINE, TIME)
-q	Quick mode — just show usernames and a count
-u	Show idle time and process ID too
am i	who am i → shows only your own login info

EXAMPLE

```
$ who -H
USER     LINE     TIME
indra    tty01    Jun 27 14:00
rahul    tty02    Jun 27 14:10

$ who am i
indra    tty01    Jun 27 14:00
```

> **Analogy:** Like a hostel warden's register — who shows everyone currently checked in, while who am i is you flipping to just your own entry.

5. bc — Basic Calculator

SYNTAX

`bc [options] [file]`

WORKING

bc opens an interactive arbitrary-precision calculator. You type expressions and it evaluates them line by line. It also supports variables, loops, and conditionals like a small programming language. Type quit to exit.

COMMONLY USED FLAGS

Flag	Meaning
-l	Load the math library (enables sin, cos, sqrt, etc.)
scale=n	(set inside bc) controls number of decimal digits shown

EXAMPLE

```
$ bc
5 + 3
8
scale=2
10 / 4
2.50
quit

$ bc -l
sqrt(2)
1.41421356237309504880
```

> **Analogy:** Like opening a scientific calculator app — by default it does basic math, but switch on the "scientific mode" (-l) to unlock square roots and trig functions.

6. echo — Display a Line of Text

SYNTAX

`echo [string]`

WORKING

echo simply prints whatever text or variable value you give it to the screen, followed by a newline. It's heavily used inside shell scripts to display messages or debug variable values.

COMMONLY USED FLAGS

Flag	Meaning
-n	Do not print the trailing newline
\c	(inside string) Stops output at that point, no newline

EXAMPLE

```
$ echo "Hello Indra"
Hello Indra

$ echo -n "No newline here"
No newline here$

$ name="Indra"
$ echo "Hi $name, welcome back"
Hi Indra, welcome back
```

> **Analogy:** Like a megaphone — you speak text into it, it shouts the exact same text back out, no questions asked.

7. zip — Compress and Bundle Files

SYNTAX

`zip [options] archive.zip file1 file2 ...`

WORKING

zip takes one or more files (or even whole directories) and bundles them together into a single compressed .zip archive, saving disk space and making it easy to transfer multiple files as one unit.

COMMONLY USED FLAGS

Flag	Meaning
-r	Recursively zip an entire directory
-e	Encrypt the archive with a password
-9	Use maximum compression (slower, smaller file)
-v	Verbose — show progress/details

EXAMPLE

```
$ zip notes.zip unit5.txt unit6.txt
  adding: unit5.txt (deflated 45%)
  adding: unit6.txt (deflated 50%)

$ zip -r project.zip myproject/
```

> **Analogy:** Like packing several separate clothes into one suitcase and compressing the air out — easier to carry, less space used.

8. unzip — Extract Files from a Zip Archive

SYNTAX

`unzip [options] archive.zip`

WORKING

unzip reverses what zip did — it reads a .zip archive and restores the original files/folders out of it, into the current directory (or a specified one).

COMMONLY USED FLAGS

Flag	Meaning
-l	List contents of the archive without extracting
-d dir	Extract into a specific directory
-o	Overwrite existing files without asking
-q	Quiet mode — suppress messages

EXAMPLE

```
$ unzip notes.zip
Archive:  notes.zip
  inflating: unit5.txt
  inflating: unit6.txt

$ unzip -l notes.zip
```

> **Analogy:** Like unpacking that same suitcase back out — taking each piece of clothing out and placing it where it belongs.

9. gzip — Compress a File

SYNTAX

`gzip [options] filename`

`gunzip filename.gz`

WORKING

gzip compresses a single file at a time, replacing the original with a .gz version (smaller in size). Unlike zip, it normally works on one file only and doesn't bundle multiple files together. gunzip reverses this, restoring the original file.

COMMONLY USED FLAGS

Flag	Meaning
-v	Verbose — show compression % achieved
-d	Decompress (same as using gunzip)
-k	Keep the original file
-9	Best compression (slowest)
-1	Fastest compression (least compressed)

EXAMPLE

```
$ gzip -v unit5.txt
unit5.txt:  62.3% -- replaced with unit5.txt.gz

$ gunzip unit5.txt.gz
```

> **Analogy:** Like vacuum-sealing one single garment tightly in a bag — great for one item at a time.

10. tar — Archiving (Tape Archive)

SYNTAX

`tar [options] archive_name file1 file2 ...`

WORKING

tar bundles multiple files and directories into a single .tar archive file, preserving structure and permissions — but by default it does not compress. It's commonly combined with gzip to both archive and compress (.tar.gz).

COMMONLY USED FLAGS

Flag	Meaning
-c	Create a new archive
-x	Extract files from an archive
-t	List contents of archive
-v	Verbose — show filenames
-f	Must be followed by the archive filename
-z	Use gzip compression along with archiving

EXAMPLE

```
$ tar -cvf backup.tar unit5.txt unit6.txt
unit5.txt
unit6.txt

$ tar -tvf backup.tar
$ tar -xvf backup.tar

$ tar -czvf backup.tar.gz unit5.txt unit6.txt
```

> **Analogy:** tar is like packing many separate files into one big envelope (archiving). Add -z and it's like sealing that envelope tightly too (compression).